Implementation Plan, PMP, WFA Classification, and the RDMA-Compute Interface

Adaptive RDMA Optimization for Distributed LLM Training and Inference on Consumer Hardware

Nathan Gopee
MS Computer Science Candidate, SUNY New Paltz
gopeen1@newpaltz.edu
Advisors: Dr. Wafi Danesh & Dr. Ashley Suchy

March 31, 2026

Contents

1 Problem Context **3**
1.1 Three Resources, One Bottleneck . . . 3
1.2 Prefill and Decode Are Different Problems . . . 3
1.3 The KV Cache: A Small Price for Linear Complexity . . . 3
1.4 The Network Becomes the Fourth Resource . . . 4
1.5 The Gap This Thesis Fills . . . 4

2 Theoretical Foundation **4**
2.1 The Scaling Book's Decode Latency Model . . . 4
2.2 PMP: Bang-Bang QP Allocation . . . 5
2.3 WFA vs. PMP: Complementary Roles . . . 5

3 Implementation Plan **5**
3.1 What We Are Building (and Not Building) . . . 6
3.2 The Three API Layers . . . 6

4 Deployment Configurations: Where RDMA Enters the Data Path **7**
4.1 Configuration A: One User, One Model, Multiple GPUs on One Machine . . . 7
4.2 Configuration B: Multiple Users, One Model, One Machine . . . 7
4.3 Configuration C: Disaggregated Prefill/Decode Across Machines . . . 8
4.4 Configuration D: Multiple Users, One Model, Multiple Machines . . . 9
4.5 Configuration E: Multiple Models, Multiple Machines . . . 9
4.6 Summary: QP and Verbs Mapping by Configuration . . . 9

5 UCX Issue Landscape and libscuffedrdma's Position **10**
5.1 Protocol Transition Cliffs: The Core Problem . . . 10
5.2 Head-of-Line Blocking in Shared QP Pools . . . 11
5.3 Transport Selection Opacity . . . 12
5.4 Multi-Rail Failures . . . 13

5.5 Rendezvous Protocol Bugs and Optimizations . . . 13
5.6 Memory Registration Overhead . . . 14
5.7 GPU and ML Workload Friction . . . 14
5.8 Congestion Control and QoS Gaps . . . 14
5.9 Summary: Where libscuffedrdma Intersects UCX's Limitations . . . 15

6 KV Cache Compression Pipeline and TurboQuant 15
6.1 Three Compression Stages on Orthogonal Axes . . . 15
6.2 TurboQuant . . . 16
6.3 How Compression and Transport Interact . . . 16

7 Production Systems Context 16

8 Benchmark Results 17
8.1 Dual QP Pool: Head-of-Line Blocking . . . 17
8.2 Latency Across Message Sizes . . . 18
8.3 Infrastructure Validation . . . 18

9 Implementation: Python MVP on SoftRoCE 18
9.1 RDMA Connection Lifecycle . . . 19
9.2 Dual QP Pool . . . 19
9.3 WFA Classifier . . . 20
9.4 PMP Bang-Bang Controller . . . 21
9.5 How RDMA Testing Works on SoftRoCE . . . 21

10 Next Steps 22

1 Problem Context

1.1 Three Resources, One Bottleneck

Every GPU balances three resources: compute (how many FLOPs/s the tensor cores can deliver), memory bandwidth (how fast data moves between VRAM and the compute units), and total memory (how much fits on the card at once). An RTX 5090 has roughly 105 TFLOPS of FP16 tensor core compute and about 1.8 TB/s of GDDR7 bandwidth. An RTX 3090 has roughly 36 TFLOPS FP32 (71 TFLOPS FP16 tensor) and 936 GB/s of GDDR6X bandwidth. The ratio of compute to bandwidth determines a critical threshold: if an operation does not perform enough FLOPs per byte loaded from memory, the tensor cores sit idle waiting for data. This is the roofline model [1], and it governs every layer of the inference stack.

1.2 Prefill and Decode Are Different Problems

LLM inference splits into two phases with fundamentally different bottlenecks. Prefill processes the full user prompt in one forward pass. Because many tokens share the same weight matrices, arithmetic intensity is high and the GPU is compute-bound. Decode generates tokens one at a time. Each step must reload the full KV cache from VRAM to compute a single attention dot product per head, producing an arithmetic intensity near 1. The GPU is memory-bandwidth-bound during decode regardless of batch size for the attention operation, and the MLP is bandwidth-bound unless the batch exceeds a hardware-specific critical threshold [1].

This asymmetry is why disaggregated serving exists: run prefill on one set of GPUs optimized for compute throughput, run decode on another set optimized for memory bandwidth, and transfer the KV cache between them.

1.3 The KV Cache: A Small Price for Linear Complexity

Without a KV cache, every decode step would recompute attention over the entire sequence from scratch–quadratic in sequence length. The KV cache makes decode linear: each new token only computes its own key and value projections, then attends over the cached keys and values from all previous tokens. This is what makes autoregressive generation practical at long context lengths.

The cost is memory. For a model with L layers, K KV heads, head dimension H, sequence length S, and 2 bytes per element (FP16):

$$\text{KV cache per request} = 2 \times L \times K \times H \times S \times 2 \text{ bytes}$$

For a 70B-class model (L=80, K=8 with GQA, H=128, S=4096): that is $2\times80\times8\times128\times4096\times2 \approx$ 1.3 GB per request. With 16 concurrent users, the KV cache alone consumes 21 GB–more than the weights on many consumer GPUs. On our RTX 3090s (24 GB VRAM each), the KV cache competes directly with model weights for memory.

This is why KV cache compression (LoRC, KVTC, TurboQuant) matters so much for this thesis: every byte saved in the cache is a byte that does not need to cross the 10GbE link during disaggregated serving, and a byte freed in VRAM for larger batches or longer contexts. Compression directly reduces the bytes that must cross the 10GbE link during disaggregated serving. Combined with the dual QP pool (which eliminates queuing delay on top of the raw transfer time), the two optimizations target independent terms in the transfer latency–one shrinks the payload, the other prevents scheduling delays. The actual combined impact is an experimental target for the Cerberus/Chimera testbed.

1.4 The Network Becomes the Fourth Resource

When the KV cache must cross a network to reach the decode GPU, a fourth resource enters: network bandwidth. On enterprise hardware, NVLink provides hundreds of GB/s between GPUs on the same node, and InfiniBand provides 25–50 GB/s between nodes. These are fast enough that many distributed operations remain compute-bound even after accounting for communication.

On the consumer hardware in this thesis (Cerberus with 2x RTX 5090, Chimera with 3x RTX 3090, connected via 10GbE at roughly 1.2 GB/s effective throughput), the network roofline drops by two orders of magnitude compared to InfiniBand. Operations that are compute-bound in a datacenter become communication-bound on this testbed. A KV cache transfer that takes milliseconds on InfiniBand takes seconds on 10GbE. An AllReduce that overlaps with computation on NVLink dominates the step time on PCIe across machines.

No GPU vendor designs their interconnect for this scenario. NVLink is intra-node only. InfiniBand requires enterprise NICs and switches. The network constraint at the consumer price point is a hardware limitation that software must compensate for.

1.5 The Gap This Thesis Fills

Existing RDMA transport libraries (UCX, NIXL, NCCL) treat all traffic equally or classify it by message size alone. UCX's eager/rendezvous threshold splits traffic at a static byte boundary. NCCL optimizes collective operations but provides no mechanism to prioritize one collective over another. Neither system knows that a small AllReduce fragment carrying a decode activation is more latency-critical than a large KV cache block being migrated for prefetch.

The thesis introduces **tensor-aware RDMA traffic management**: a middleware that classifies each transfer by its role in the inference pipeline (via the WFA), determines optimal bandwidth allocation between priority classes (via PMP's bang-bang controller), and routes traffic to physically separated RDMA queue pairs (hot pool for latency-sensitive, cold pool for throughput-oriented). This is a software solution to a hardware limitation, and it produces larger relative gains on consumer interconnects precisely because the network is so constrained that every wasted microsecond of head-of-line blocking is measurable.

2 Theoretical Foundation

2.1 The Scaling Book's Decode Latency Model

Austin et al. [1] derive the central equation connecting KV cache size to decode latency. During decode, the theoretical minimum step time is:

$$T_{\text{step}} = \underbrace{\frac{B \times \text{KV}_{\text{size}}}{W_{\text{HBM}}}}_{\text{attention (always BW-bound)}} + \underbrace{\max\left(\frac{2B \times P}{C_{\text{FLOPs}}}, \frac{P_{\text{bytes}}}{W_{\text{HBM}}}\right)}_{\text{MLP (can be compute-bound)}} \tag{1}$$

The attention term has no compute roofline because decode attention's arithmetic intensity is approximately 1 (one dot product per KV byte loaded from HBM), so it is always memory-bandwidth-bound. The MLP term transitions at $B_{\text{crit}} = C_{\text{FLOPs}}/W_{\text{HBM}}$ [1]. For disaggregated serving, TTFT adds the network transfer:

$$T_{\text{TTFT}} = T_{\text{prefill}} + T_{\text{KV transfer}} + T_{\text{first decode step}} \tag{2}$$

The thesis targets $T_{\text{KV transfer}}$. On a bandwidth-bound 10GbE link, $T_{\text{KV transfer}} = \text{KV}_{\text{bytes}}/W_{\text{net}}$. Every optimization maps to a specific term:

Tool	Term reduced	Mechanism
Triton / Pallas / K-Search	T_{prefill}, $T_{\text{decode step}}$	Better kernel → closer to compute roofline
GQA / MQA [26, 27]	KV_{size} in Eq. 1, 2	Fewer KV heads → smaller cache per layer
LoRC [8]	KV_{size}	SVD on W_K, W_V → lower-dim KV vectors
KVTC [9] / TurboQuant [10]	KV_{bytes}	Quantize + entropy-code the KV tensors
Dual QP pool	$T_{\text{KV transfer}}$	Prevent head-of-line blocking on 10GbE
PMP controller	$T_{\text{KV transfer}}$	Optimal bandwidth allocation across QPs
PTX warp specialization [14]	T_{prefill} overlap	$\max(T_{\text{math}}, T_{\text{comms}})$ not sum
Paged Attention [29]	KV_{bytes}	Transfer only populated pages, not padded
Layer-by-layer xfer [15]	$T_{\text{KV transfer}}$ overlap	Pipeline prefill layers with RDMA writes

Table 1: Each optimization maps to a specific term in the TTFT equation.

2.2 PMP: Bang-Bang QP Allocation

Zhang et al. [2] reinterpret LoRA matrices as control variables on a dynamical system. PMP [3, 4] provides the necessary condition for optimal control.

Definition 1 (RDMA Allocation as Controlled System)**.** *Let* $\mathbf{s}(t) = (q_H(t), q_C(t))$ *be hot/cold queue depths. Control* $u(t) \in [0, 1]$ *sets the hot-path bandwidth fraction:*

$$\dot{q}_H = \lambda_H(t) - u(t) \cdot C \cdot \mu_H, \qquad \dot{q}_C = \lambda_C(t) - (1 - u(t)) \cdot C \cdot \mu_C \tag{3}$$

where λ_H, λ_C *are WFA-classified arrival rates and* C *is 10GbE link capacity.*

Proposition 1 (Bang-Bang QP Allocation)**.** *The Hamiltonian* $\mathcal{H} = p_H \dot{q}_H + p_C \dot{q}_C$ *is linear in* u*, so optimal allocation is bang-bang:* $u^*(t) = 1$ *if* $S > 0$*,* $u^*(t) = 0$ *if* $S < 0$*, where*

$$S = p_H \cdot C \cdot \mu_H - p_C \cdot C \cdot \mu_C$$

is the switching function and p_H, p_C *are the costate (adjoint) variables satisfying* $\dot{p}_i = -\partial\mathcal{H}/\partial q_i$*. In the implementation, we approximate the costates by weighted queue depths:* $p_H \approx \alpha \cdot q_H$*,* $p_C \approx \beta \cdot q_C$*, giving the practical switching function* $S = \alpha \cdot q_H \cdot C \cdot \mu_H - \beta \cdot q_C \cdot C \cdot \mu_C$*.*

2.3 WFA vs. PMP: Complementary Roles

	WFA Classifier	PMP Controller
Question	Which class is this tensor?	How much bandwidth per class?
Output	hot / warm / cold label	$u^*(t) \in \{0, 1\}$
Timescale	Per-tensor (μs)	Per-scheduling-epoch (ms)

3 Implementation Plan

This section describes the architecture of libscuffedrdma, the thesis's C++ RDMA middleware with pybind11 bindings. Austin et al.'s *How to Scale Your Model* [1] serves as the analytical reference throughout: its roofline framework, decode step time equation, and parallelism taxonomy directly inform every design decision below.

3.1 What We Are Building (and Not Building)

We are not recreating NIXL or UCX. We are building a layer that sits between vLLM's `KvConnector` interface and the raw `libibverbs` API, adding tensor-aware traffic classification and priority QP routing that neither NIXL nor UCX provides. The full stack:

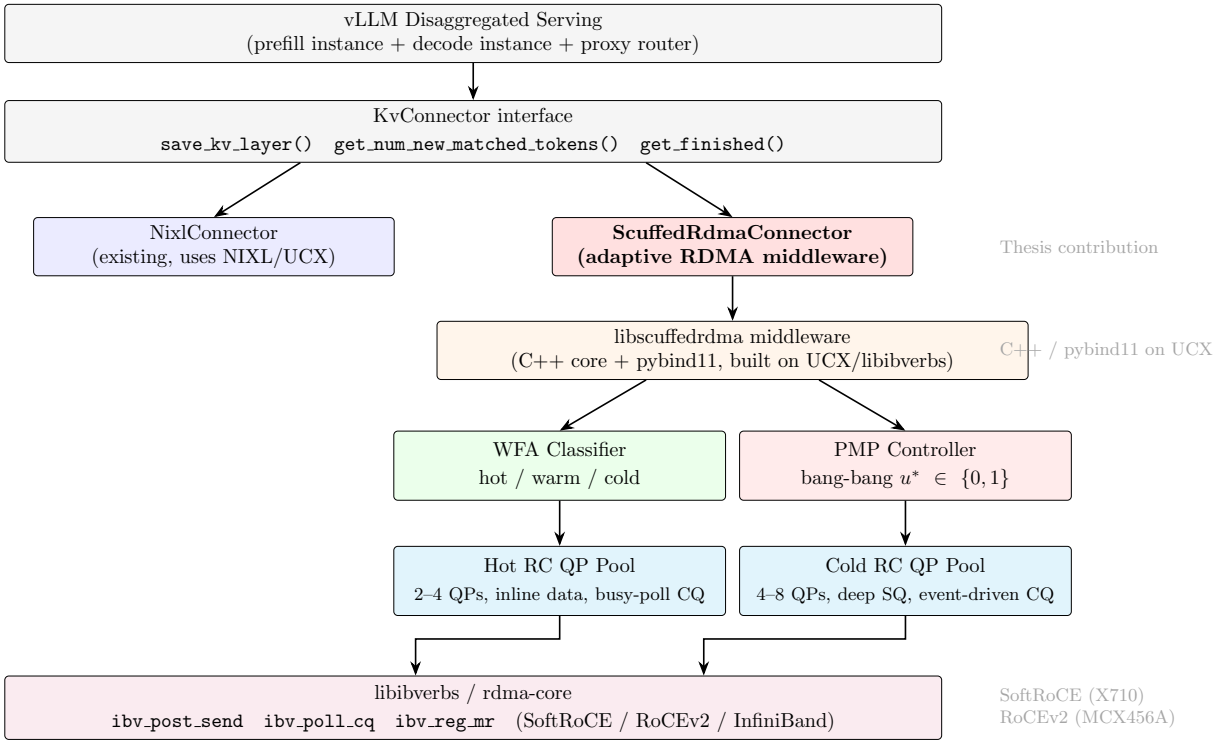

Figure 1: Full system architecture. The middleware (red) adds tensor-aware classification and priority QP routing on top of UCX/libibverbs, replacing NixlConnector's transport layer. Everything above `KvConnector` stays stock vLLM.

3.2 The Three API Layers

libscuffedrdma bypasses UCX's transport selection and calls `ibv_post_send` directly. UCX routes messages by size alone (small = eager inline send, large = rendezvous RDMA read). This means UCX cannot distinguish between a latency-critical 64KB decode activation and a 64KB background prefetch. libscuffedrdma's WFA classifier makes that distinction, then posts to the correct QP.

The existing vLLM call chain goes: `save_kv_layer()` → NIXL → UCX → `ibv_post_send()`. We replace the middle: `save_kv_layer()` → WFA classifies → `ibv_post_send()` on the selected QP.

Both QP pools use RC (Reliable Connected) transport. We do not use UD (Unreliable Datagram) because UD has a 4KB MTU limit requiring application-level fragmentation, no delivery guarantees, and poor SoftRoCE performance. RC handles reliability in hardware. The cost is one QP per peer per priority class, which is trivial for a two-node testbed.

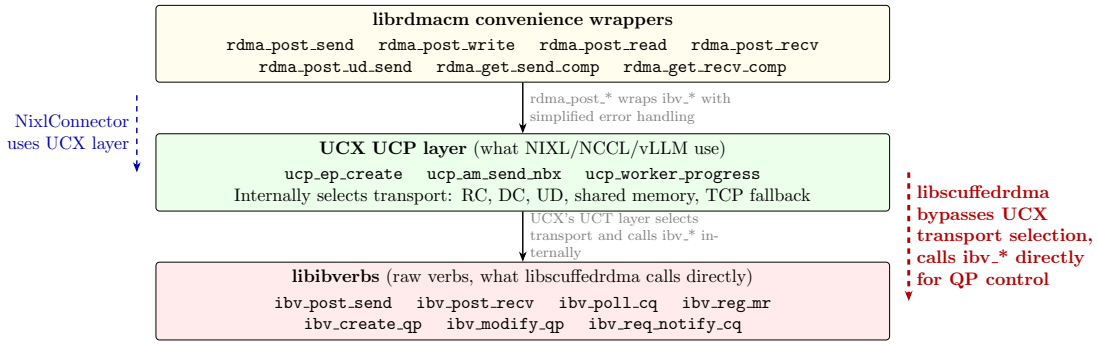

Figure 2: Three RDMA API layers. NixlConnector calls UCX, which internally selects transports. libscuffedrdma bypasses UCX's transport selection and calls `ibv_*` directly, which is necessary for controlling which QP each transfer uses.

	Hot Pool (latency)	**Cold Pool (throughput)**
SQ depth	32–64 entries	256–512 entries
Inline data	Enabled ($\leq$256B in WR)	Disabled (DMA from MR)
CQ polling	Busy-poll, dedicated core	Event-driven (`ibv_req_notify_cq`)
Signaling	Every WR	Every Nth WR
Traffic	KV metadata, heartbeats, all-reduce	Bulk KV blocks, weights, gradients

Table 2: Hot vs. cold RC QP pool configuration. Both pools use RC transport; they differ in tuning.

4 Deployment Configurations: Where RDMA Enters the Data Path

The following configurations cover the realistic deployment scenarios for distributed LLM inference. In each case we trace four data structures through the system: **model weights** (static, loaded once), **activations** (ephemeral, per-forward-pass), **KV cache** (grows per-token, persists per-request), and **control signals** (metadata, scheduling, consensus). The parallelism strategies below follow directly from Austin et al. [1] Sections 5 and 7.

4.1 Configuration A: One User, One Model, Multiple GPUs on One Machine

This is **tensor parallelism** (TP). Each layer's weight matrices are sharded across GPUs. Every GPU holds a slice of every layer. A single user's request flows through all GPUs at every layer.

Each GPU computes its shard of the attention and MLP, then AllReduces the partial result with every other GPU. This happens at every layer. The AllReduce messages are small (a few KB to hundreds of KB) but every GPU blocks waiting for them. This is textbook **hot QP** traffic: small, frequent, latency-critical. On consumer hardware without NVLink, AllReduce goes over PCIe or 10GbE and becomes the bottleneck. vLLM falls back to NCCL for inter-node AllReduce when GPU IPC is unavailable, which is where libscuffedrdma enters.

4.2 Configuration B: Multiple Users, One Model, One Machine

Same as Config A but with continuous batching ($B > 1$ users). Each user has their own KV cache, so total KV memory per GPU scales as $B \times L \times 2 \times K \times H/N$. The RDMA traffic is the same

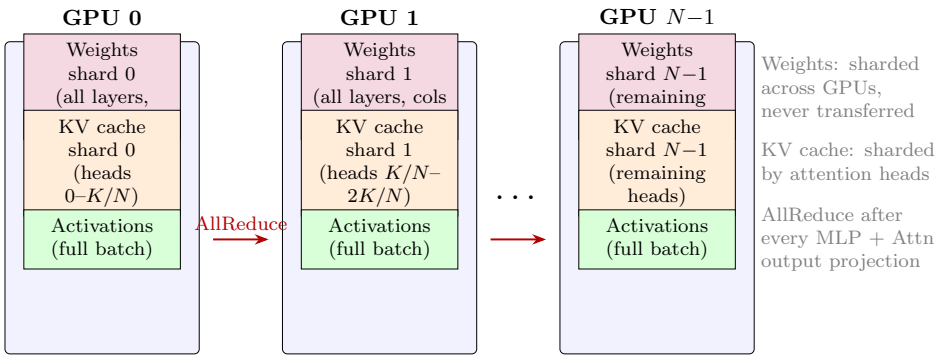

Figure 3: **Config A:** Tensor parallelism, single machine. AllReduce happens over NVLink (intra-node) or PCIe. On consumer hardware without NVLink, PCIe AllReduce is the bottleneck.

(AllReduce on hot QP) but payloads grow with batch size.

Figure 4: **Config B:** Same as A with B users batched. KV caches grow with batch size.

4.3 Configuration C: Disaggregated Prefill/Decode Across Machines

This is the primary target of the thesis. Prefill and decode run on **separate machines**. The prefill node processes the full prompt and generates the KV cache. The KV cache is then transferred over the network to the decode node, which generates tokens autoregressively.

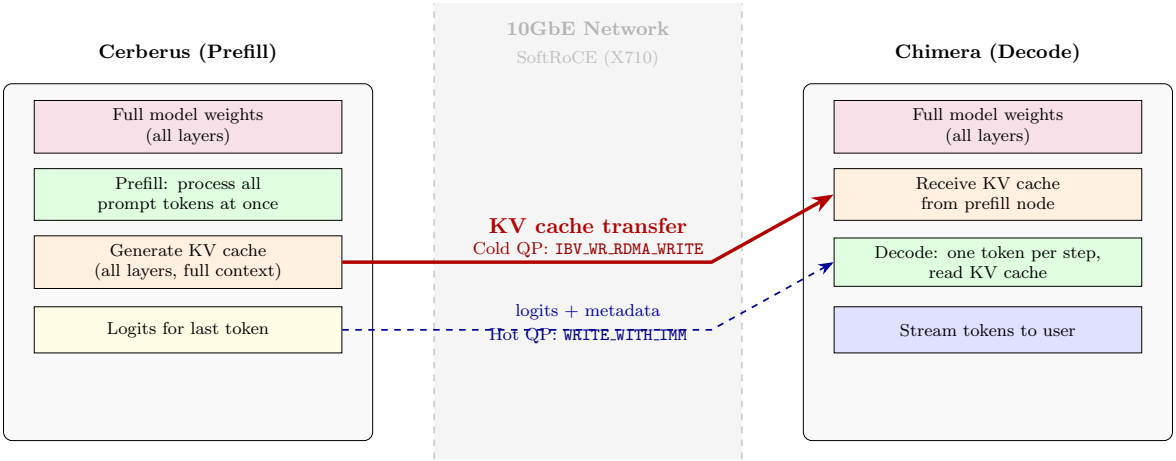

Figure 5: **Config C:** Disaggregated prefill/decode. The thick red arrow is the critical-path RDMA transfer that determines TTFT. The dashed blue arrow carries latency-sensitive metadata on the hot QP.

Data flow. The prefill node holds a complete copy of model weights. It processes the full prompt in one forward pass (compute-bound, high arithmetic intensity). It generates the KV cache for all

layers and the logits for the last token. Both are transferred to the decode node via RDMA. The decode node also holds a complete copy of model weights. It receives the KV cache, appends one token's KV per step, and generates tokens autoregressively (memory-bandwidth-bound).

What crosses the network. The KV cache is the dominant transfer. For a model with L layers, K KV heads, head dimension H, and sequence length S:

$$\text{KV transfer size} = 2 \times \text{bytes} \times H \times K \times L \times S$$

The logits and request metadata are small (a few KB). On 10GbE, the KV transfer is the TTFT bottleneck.

Dual QP mapping. **Hot QP:** logits, request metadata, scheduling signals, Raft heartbeats (if running consensus for multi-node coordination). These are small and latency-critical: the decode node cannot begin until it has the logits. **Cold QP:** bulk KV cache blocks. These are large but can be pipelined layer-by-layer (TransferEngine pattern [15]). Using `IBV_WR_RDMA_WRITE` for zero-copy one-sided transfer from prefill GPU memory to decode GPU memory (via GPUDirect RDMA on the V100's MCX456A, or staged through host memory on Cerberus/Chimera with SoftRoCE).

Compression reduces the cold QP load. LoRC [8] reduces H (lower-dimensional KV vectors). KVTC [9] applies entropy coding on the KV tensors. TurboQuant [10] reduces bytes-per-element. Each shrinks the cold QP payload, but the hot QP traffic (metadata, logits) is unaffected since it is already small.

4.4 Configuration D: Multiple Users, One Model, Multiple Machines

This combines Configs B and C: disaggregated prefill/decode with continuous batching across multiple users.

Multiple users' KV transfers share the same 10GbE link. Without traffic separation, User A's large KV transfer blocks User B's small metadata. This is the core head-of-line blocking scenario the dual QP pool solves. The PMP controller dynamically allocates bandwidth between pools based on queue depth. With prefix caching (llm-d [17]), some KV blocks are already on the decode node, changing the traffic pattern the WFA observes.

4.5 Configuration E: Multiple Models, Multiple Machines

MoE (Mixture-of-Experts) models and multi-model serving.

In MoE models (e.g. DeepSeek-V3), each token routes to a subset of experts across machines, creating All-to-All traffic at every layer. Popular experts get large predictable batches (cold QP), rare experts get small urgent transfers (hot QP). The WFA classifies by per-expert access frequency. vLLM's elastic expert parallelism (PR #34861 [21]) creates this pattern during dynamic GPU scaling.

4.6 Summary: QP and Verbs Mapping by Configuration

What to optimize first. Config C (disaggregated prefill/decode, single user) is the simplest configuration that exercises the full dual QP architecture. It has a clear hot/cold separation (metadata vs. KV blocks), a measurable TTFT metric, and maps directly onto the Cerberus/Chimera

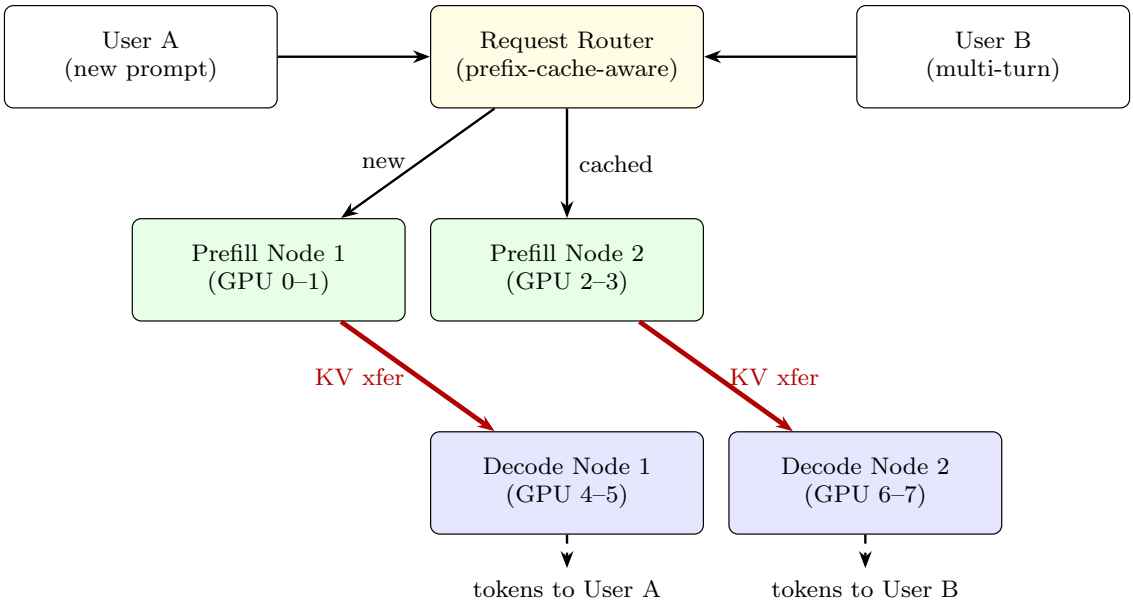

Figure 6: **Config D:** Multi-user disaggregated serving. The router directs requests to prefill nodes based on prefix cache locality (llm-d pattern [17]). KV transfers (red) are independent per-user and can be multiplexed on the same link.

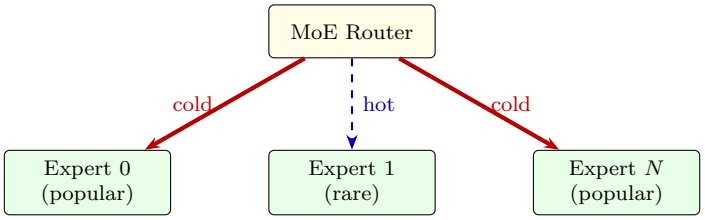

Figure 7: **Config E:** MoE expert parallelism. Popular experts get large batches (cold QP), rare experts get small urgent transfers (hot QP).

testbed. Once Config C works, extending to Config D (multiple users) adds the multiplexing problem that makes the PMP controller's bang-bang allocation necessary. Config E (MoE) is the most complex but produces the richest WFA training data due to the bimodal expert access pattern.

5 UCX Issue Landscape and libscuffedrdma's Position

UCX is the dominant open-source RDMA transport framework, used by NCCL, UCC, MPI implementations, and distributed ML frameworks including PyTorch and Dask. An analysis of the UCX issue tracker (`https://github.com/openucx/ucx/issues`) reveals recurring classes of problems that stem from architectural decisions libscuffedrdma is explicitly designed to avoid. This section maps those issues to the thesis contributions.

5.1 Protocol Transition Cliffs: The Core Problem

UCX routes messages based on size alone. Messages below `UCX_RNDV_THRESH` (default around 8 KB) take the eager path (inline send, low latency); messages above take the rendezvous path (zero-copy

Config	Hot QP traffic	Cold QP traffic	Primary verb
A: TP, 1 user, 1 machine	AllReduce fragments	(none)	`WRITE_WITH_IMM`
B: TP, N users, 1 machine	AllReduce fragments	(none)	`WRITE_WITH_IMM`
C: Disagg P/D, 1 user	Logits, metadata	KV cache blocks	`WRITE` (cold)
D: Disagg P/D, N users	Metadata, scheduling	Multiplexed KV	`WRITE` (cold)
E: MoE / multi-model	Rare expert tokens	Popular expert tokens	`WRITE_WITH_IMM`

Table 3: QP and verbs mapping by deployment configuration. Config C is the thesis's primary target; Configs D and E are stretch goals.

RDMA read, high throughput). This creates latency cliffs at the threshold boundary:

- **Issue #10552**: On Grace Hopper systems, 65 536-byte messages exhibit **50× worse latency** than 131 072-byte messages in OSU alltoall benchmarks. The cliff occurs precisely at the eager-to-rendezvous transition, where the sender must negotiate a rendezvous handshake, register memory, and wait for the receiver to initiate an RDMA read–all adding round-trips that the eager path avoids.
- **Issue #10486**: Java/JUCX applications observe severe performance degradation for messages above 32 KB when UCX switches from RDMA SEND (eager) to RDMA READ (rendezvous). The user reports that "the performance drops off a cliff."
- **Issue #10532**: Active Message and tag latency degrade sharply after ∼30 000 bytes, at the boundary between eager and zcopy or rendezvous thresholds.
- **Issue #11091**: Upgrading from UCX 1.18.1 to 1.19.1 causes significant regressions due to changes in rendezvous performance difference calculations (PR #10401), demonstrating that the RNDV heuristic is fragile and version-sensitive.
- **Issue #4430**: `ucp_tag_send_nbr()` hardcodes a rendezvous threshold instead of using the dynamically calculated one, so even UCX's own internal APIs do not consistently apply the threshold logic.

Why libscuffedrdma avoids this. libscuffedrdma does not use message size to select a protocol. The WFA classifier examines tensor metadata–access frequency, inference phase (prefill vs. decode), layer index, and variance of inter-arrival times–to assign a priority class (hot, warm, or cold). The PMP controller then determines QP allocation. A 64 KB KV cache block for a decode step and a 64 KB prefetch block for speculative prefill are the same size but have fundamentally different latency requirements; UCX treats them identically, libscuffedrdma does not.

5.2 Head-of-Line Blocking in Shared QP Pools

UCX uses a shared pool of RC queue pairs with static partitioning. A small eager message can queue behind a large rendezvous transfer on the same QP because the QP selection has no knowledge of message priority. Several issues trace directly to this design:

- **Issue #11004**: All RoCE QPs across different processes use the **same UDP source port**, causing bond/LAG layer3+4 hashing to route all RDMA traffic through a single physical link. This is a consequence of QPs being allocated from a shared pool without per-flow differentiation. libscuffedrdma's per-classification QP pools (hot pool: 2–4 QPs, cold pool:

4–8 QPs) naturally produce different 5-tuple flow hashes, distributing traffic across bond members without requiring changes to the switch or bond configuration.

- **Issue #11034**: Eight UCP workers sharing a `ucp_context` contend on a `pthread_spin_lock` even when each operates in `THREAD_MODE_SINGLE`, preventing linear scaling. The contention arises because all workers share the same QP and CQ resources. libscuffedrdma's dedicated QPs per traffic class eliminate this shared-resource bottleneck.
- **Issue #1319**: A long-standing feature request for on-demand QP creation (deferring UCT endpoint/QP allocation until first message rather than at `ucp_ep_create` time). libscuffedrdma already implements this pattern: QPs are created per classification when the first tensor of that class is submitted, not at connection establishment.

The fundamental issue. UCX's QP pool is a shared resource managed by size-based heuristics. libscuffedrdma treats QPs as *class-dedicated resources* managed by a control-theoretic policy (PMP). A hot-classified tensor is *guaranteed* to be posted on the hot QP pool and never queued behind cold traffic, because the `ibv_post_send` call targets a specific QP selected by the WFA+PMP pipeline, not by UCX's internal dispatcher.

5.3 Transport Selection Opacity

UCX's transport selection algorithm is a black box that users cannot easily reason about or override:

- **Issue #10652**: UCX selects `mlx5`/IB transport for intra-node communication instead of the expected XPMEM or CMA single-copy mechanisms, resulting in worse latency. The user has no mechanism to force the correct transport.
- **Issue #6511**: RC transport is prioritized over `dc_x` (dynamically connected) even when DC is available and would be more appropriate. Users cannot control preference order.
- **Issue #9364**: "What is the logic of choosing the correct transport at runtime?"–users find UCX's transport selection algorithm opaque and undocumented.
- **Issue #9560**: No fine-grained per-path transport pinning in multi-node environments. UCX selects different configurations depending on communication patterns, and users lack the ability to pin specific transports per data path.
- **Issue #10608**: No API to query which transport was actually used for a given transfer. Users cannot observe the transport decision after it is made.
- **Issue #11282**: On 8×H200 with NVLink, GPU pair latency varies dramatically because UCX selects `mlx5` transport for some intra-node pairs instead of NVLink, causing inconsistent alltoallv performance.

libscuffedrdma's transparency. libscuffedrdma makes exactly one transport decision: all inter-node tensor data flows through `ibv_post_send` on a specific QP selected by the WFA classifier. The mapping from tensor metadata to QP is deterministic given the WFA state and PMP control signal $u^*(t)$. There is no opaque scoring function, no multi-transport negotiation, and no version-sensitive heuristic. UCX's connection management infrastructure (`rdma_create_id`, `rdma_resolve_addr`, or `ucp_ep_create`) is used only for the initial handshake; the data path is fully controlled by the thesis middleware.

5.4 Multi-Rail Failures

UCX's multi-rail support–intended to aggregate bandwidth across multiple NICs–consistently fails to deliver expected gains and sometimes degrades performance:

- **Issue #9520**: Two ConnectX-6 200 Gbps NICs with multi-rail enabled show **no bandwidth improvement** over a single NIC.
- **Issue #10249**: Configuring two NICs in GDR mode results in dramatically worse performance than a single NIC, suggesting path interference.
- **Issue #10669**: On quad-socket systems with per-socket NICs, UCX uses remote-socket NICs, adding cross-NUMA latency that degrades rather than improves performance.
- **Issue #10420**: Multi-rail with two bond devices causes `ucx_perftest` to hang entirely.
- **Issue #11284**: The hardcoded `UCP_MAX_LANES=64` limit prevents configurations with 8 GPUs and 9 NICs from utilizing all available bandwidth lanes.

Relevance to the thesis. On the Cerberus/Chimera testbed (single 10 GbE NIC per machine), multi-rail is not directly applicable. However, the architectural principle is relevant: libscuffedrdma's direct `ibv_post_send` with NUMA-aware QP placement avoids the lane-selection logic that causes these failures. If the testbed were extended with a second NIC, hot QPs would be pinned to the NUMA-local NIC and cold QPs to the secondary, with no reliance on UCX's lane scoring.

5.5 Rendezvous Protocol Bugs and Optimizations

The rendezvous protocol–which libscuffedrdma bypasses for tensor data–has several open issues that further motivate the bypass:

- **Issue #6721**: The first Active Message between an endpoint pair is **always received as eager** regardless of `UCX_RNDV_THRESH`, because endpoint wireup has not completed. Subsequent messages use rendezvous correctly. This creates a first-message anomaly that affects latency profiling and warmup behavior.
- **Issue #10568**: Setting `UCX_MAX_RNDV_RAILS=1` causes segfaults and hangs on multi-NIC GPU systems, indicating fragility in RNDV rail selection.
- **Issue #3542**: Rendezvous causes $\sim 3\times$ the wakeup overhead compared to eager (3 wakeup notifications vs. 1), impacting event-driven progress engines used by ucxpy/Dask.
- **Issues #1298, #1299, #1300**: Missing optimizations for `get_bcopy`, `put_bcopy`, and receiver-side buffer registration in the rendezvous path. These have been open since 2017, indicating the rendezvous implementation has accumulated technical debt.

Why bypassing RNDV is justified. libscuffedrdma bypasses the rendezvous data path. It calls `ibv_post_send` directly with pre-registered memory regions. This eliminates the rendezvous handshake, avoids rail selection bugs, and removes the wakeup overhead. The trade-off is managing memory registration directly, but LLM tensor buffers are long-lived so registration cost is paid once at startup.

5.6 Memory Registration Overhead

- **Issue #10719**: `ucp_mem_map` registers memory at only 6–13 GB/s, meaning an 80 GB GPU memory region takes ∼13 seconds to register. For LLM serving with large KV caches, this is a significant startup cost.
- **Issue #7989**: `ucp_mem_map` returns success even if the NIC memory domain fails to register CUDA memory, silently falling back to software emulation paths.
- **Issue #6264**: `ibv_reg_mr` fails with "Cannot allocate memory" during long-running simulations due to resource exhaustion.

libscuffedrdma's approach. The middleware pre-registers KV cache buffers at allocation time using `ibv_reg_mr` directly, with explicit error checking (no silent fallback). Because vLLM's PagedAttention [29] allocates KV cache blocks from a fixed pool, the registration set is known at startup and does not grow unboundedly. Registration cost is paid once, not per-transfer.

5.7 GPU and ML Workload Friction

Several issues document friction between UCX and ML workloads specifically:

- **Issue #10936**: 8×H20 GPUs with 400 GB/s theoretical NVLink bandwidth achieve only ∼80 GB/s (one-fourth of peak) in UCX performance tests.
- **Issue #9908**: Multi-node distributed training fails with any `UCX_TLS` other than `rc`, and changing `UCX_NET_DEVICES` from "all" causes failures.
- **Issue #11006**: PyTorch's distributed training creates multiple threads, but UCX's TCP transport is not thread-safe, raising correctness concerns.
- **Issue #8572**: CUDA-enabled UCX adds significant startup overhead to MPI initialization, even when CUDA is not used for the current run.
- **Issue #7880**: TCP transport is 25–75% slower when CUDA support is compiled in, due to memory type detection overhead on every buffer, even for host-memory transfers.

These issues collectively demonstrate that UCX was designed as a general-purpose communication framework and retrofitted for GPU/ML workloads, rather than designed around them. libscuffedrdma is purpose-built for tensor transport: it knows about KV cache blocks, attention layers, and inference phases because the WFA encodes this domain knowledge directly.

5.8 Congestion Control and QoS Gaps

- **Issue #11252**: Enhanced Connection Establishment (ECE) defaults do not activate ZTR-RTT hardware congestion control on mlx5 even when the hardware supports it.
- **Issue #10325**: DSCP/traffic class settings via `UCX_IB_TRAFFIC_CLASS` only apply to the local direction; the remote side sends with TC=0, breaking QoS-based routing in the fabric.

Relevance. Because libscuffedrdma calls `ibv_post_send` directly, it can set ECE bits and traffic class per QP at creation time (`ibv_modify_qp` with `IBV_QP_QKEY` and traffic class attributes). Hot QPs can be assigned a higher traffic class than cold QPs, enabling hardware-level prioritization in switches that support DSCP-based scheduling. UCX's asymmetric TC application (Issue #10325) does not affect libscuffedrdma because both endpoints are under the middleware's control.

5.9 Summary: Where libscuffedrdma Intersects UCX's Limitations

Table 4 maps the thesis contributions to the UCX issues they address.

Thesis Component	UCX Issues	Mechanism
WFA classifier (tensor-aware routing)	#10552, #10486, #10532, #4430, #11091	Replaces size-only threshold with multi-dimensional tensor metadata classification
Hot/cold QP separation	#11004, #11034, #1319	Dedicated QPs per traffic class eliminate head-of-line blocking and shared-resource contention
Direct `ibv_post_send` bypass	#6721, #10568, #3542, #1298–#1300	Avoids rendezvous handshake, rail selection bugs, and wakeup overhead
Pre-registration of KV cache buffers	#10719, #7989, #6264	Amortized `ibv_reg_mr` at allocation time with explicit error handling
PMP bang-bang controller	#9520, #10669	Control-theoretic QP allocation replaces static lane scoring
Per-QP traffic class and ECE	#11252, #10325	Symmetric TC and congestion control configuration via direct `ibv_modify_qp`

Table 4: Mapping of libscuffedrdma components to UCX issues they address.

Contribution framing. The UCX issue tracker provides empirical evidence that the problems libscuffedrdma targets are not theoretical: they cause measurable performance regressions, hangs, and incorrect behavior in production deployments. The thesis does not claim to fix UCX; it claims that for the specific workload of distributed LLM inference on consumer hardware, bypassing UCX's data path and replacing it with tensor-aware, control-theoretic QP management addresses architectural limitations that UCX's existing configuration knobs cannot.

6 KV Cache Compression Pipeline and TurboQuant

6.1 Three Compression Stages on Orthogonal Axes

LoRC [8] applies SVD to W_K, W_V, producing lower-dimensional KV vectors at every forward pass without retraining. The progressive strategy retains more singular values in shallow layers (sensitive to truncation) and compresses deep layers aggressively. Zhang et al. report significant compression with negligible quality degradation across LLaMA-8B through 70B [8]. The degree to which this translates to TTFT improvement on our testbed is an open experimental question.

LoRC, KVTC [9], and TurboQuant [10] target orthogonal representation axes and compose as a pipeline. The pipeline for a KV vector $k \in \mathbb{R}^d$ is:

$$k \xrightarrow{\text{LoRC (dim)}} k' \in \mathbb{R}^{d'} \xrightarrow{\text{KVTC (redundancy)}} \tilde{k} \in \mathbb{R}^{d'} \xrightarrow{\text{TurboQuant (precision)}} \hat{k} \in \{0, \ldots, 2^b - 1\}^{d'}$$

LoRC reduces $d \to d'$ via SVD on the weight matrices. KVTC applies PCA decorrelation and entropy coding on the generated vectors. TurboQuant reduces the bits-per-element from 16 to as low as 3 while preserving inner product accuracy. Because each stage operates on a different axis of the representation, their effects should compound when combined.

6.2 TurboQuant

TurboQuant [10] (Zandieh, Mirrokni et al., Google Research) is a two-stage quantization algorithm that does not need per-block calibration data:

1. **PolarQuant**: Apply a random rotation to the KV vector, then quantize. The rotation makes each coordinate's distribution predictable, so the quantizer can be pre-computed once and reused for all vectors. No per-block normalization constants needed.
2. **QJL correction**: The quantization error creates bias in inner products. TurboQuant corrects this by storing a 1-bit sketch of the residual using the Quantized Johnson-Lindenstrauss transform. The corrected inner products are provably unbiased.

The important detail for RDMA: TurboQuant preserves *inner product accuracy*, not individual vector accuracy. Attention only needs inner products (query dot key), so this is fine. But it means the decode node needs a modified attention kernel that works directly on the quantized codes rather than decompressing first. The RDMA layer just moves the smaller compressed bytes.

6.3 How Compression and Transport Interact

Compression (TurboQuant, KVTC, LoRC) reduces the bytes that cross the link. The dual QP pool reduces the queuing delay on the link. These target different terms:

$$T_{\text{KV transfer}} = \underbrace{\frac{\text{KV}_{\text{bytes}}}{W_{\text{net}}}}_{\text{compression reduces this}} + \underbrace{T_{\text{queuing delay}}}_{\text{dual QP reduces this}}$$

The RDMA layer does not care whether it is moving compressed or uncompressed bytes. The WFA classifies by size and access pattern, not data content. However, compression shifts the size distribution: blocks that were "large" (cold path) may become "small" (hot path) after compression, so the WFA thresholds should be tuned on compressed traffic.

KVTC (NVIDIA) requires per-model calibration but achieves higher compression. TurboQuant (Google) is model-independent but compresses less. They operate at different pipeline stages and can be combined. The thesis contribution is orthogonal to both.

7 Production Systems Context

NVIDIA Dynamo/NIXL [11]: NIXL selects backends automatically but lacks adaptive QP management. libscuffedrdma fills this gap.

llm-d v0.5 [17]: UCCL's software transport with fine-grained flow splitting provides greater congestion resilience than UCX [18] by avoiding hardware PFC. Their prefix-cache-aware scheduling reduces TTFT by routing to nodes holding the relevant KV cache.

TransferEngine [15]: Bridges common NICs to expose a uniform interface for point-to-point RDMA in LLM systems, implementing one-sided WriteImm operations across multiple NICs per GPU. libscuffedrdma extends this with tensor-aware classification for QP selection.

Consensus protocols [25]: Raft heartbeats are a textbook hot-path case. On a single 10GbE NIC, a heartbeat blocked behind a large KV transfer triggers spurious leader elections.

Consumer network roofline [1]: On 10GbE ($\sim$1.2 GB/s), operations that are compute-bound in datacenters (all-reduce, KV transfer) become communication-bound. Intelligent software RDMA management produces larger relative gains on consumer hardware than on enterprise hardware where raw bandwidth masks inefficiencies. This is the core thesis argument.

8 Benchmark Results

The following results were collected on Chimera (3$\times$ RTX 3090, Aquantia AQC107 10GbE, SoftRoCE/rxe0, Ubuntu 24.04, CUDA 12.4, Python 3.12, pyverbs 50.0). All measurements use real RDMA Write operations over the rxe kernel driver.

8.1 Dual QP Pool: Head-of-Line Blocking

Table 5: Small-transfer latency under mixed 256B + 1MB workload (1000 iterations, SoftRoCE loopback).

Scenario	p50 (μs)	p95 (μs)	p99 (μs)
Single QP (baseline)	13.6	15.5	19.7
Dual QP (WFA classifier)	14.4	17.1	21.4
Dual QP + PMP controller	14.7	17.7	23.8

The dual QP scenarios are 1–4μs slower than the single QP baseline. This is expected for two reasons:

Why dual QP is slower here. SoftRoCE is synchronous and single-threaded: each transfer posts, the kernel processes it, completion arrives, then the next transfer starts. Nothing is ever queued behind anything else, so there is no head-of-line blocking to eliminate. The overhead comes from managing two pools instead of one: round-robin QP selection, building separate GlobalRoute and AHAttr structures per QP, running the WFA classifier (access count lookup, phase detection, threshold checks), and running the PMP controller (switching function, history tracking). These are the costs of the classification and routing logic.

Why this changes with real hardware. SoftRoCE implements RDMA in the kernel. The CPU handles every QP state transition, packet construction, and completion notification in software. On a hardware NIC (e.g. ConnectX-6), the NIC itself processes the send queue and generates completions independently of the CPU. This changes two things:

- **Base latency drops.** Hardware RDMA on ConnectX typically achieves 1–2μs for small messages vs our 13–14μs on SoftRoCE. The classification overhead (1–4μs) becomes a larger fraction of the transfer time but is still small relative to the cold path.
- **QP isolation becomes real.** With hardware offload, the NIC processes each QP's send queue independently. A 1MB write occupying the cold QP does not block the hot QP because the NIC has separate hardware contexts for each. With SoftRoCE, the kernel processes everything sequentially regardless of which QP it belongs to. Hardware RDMA is where the dual QP architecture pays off: concurrent producers can post to hot and cold QPs simultaneously, and the NIC drains them in parallel.

When head-of-line blocking appears. The benefit requires at least one of: (a) multiple threads posting transfers concurrently, so one thread's 1MB cold write is in-flight while another thread's 256B hot write needs to go out; or (b) async posting with queue depth > 0, where multiple work requests are queued before any complete. In both cases, sharing a single QP means the 256B message waits behind the 1MB message in the send queue. Separate QPs eliminate this wait. The concurrent workload benchmark and cross-node testing on real hardware are the next steps to measure this.

8.2 Latency Across Message Sizes

Table 6: libscuffedrdma latency vs. message size (50 iterations per size, SoftRoCE loopback). Sizes marked † correspond to UCX eager/RNDV protocol transition boundaries.

Size (B)	p50 (μs)	p95 (μs)	p99 (μs)	Pool
64	12.6	19.4	48.1	hot
256	12.9	14.5	24.6	hot
1024	12.8	14.3	24.8	hot
3000†	18.7	22.3	30.8	hot
8192†	32.7	38.1	44.5	hot
32768†	98.4	109.4	112.8	hot
65536†	183.8	207.1	278.2	hot
262144	704.9	731.5	742.3	hot
524288	1414.5	1440.7	1459.1	cold
1048576	2822.6	2870.5	3104.0	cold

The latency curve is smooth and monotonic across all message sizes. There is no cliff at the 8KB, 30KB, 32KB, or 64KB boundaries where UCX switches between eager, zcopy, and rendezvous protocols (issues #10552, #10486, #10532). The WFA classifier transitions from hot to cold pool at 512KB, matching the configured threshold of 256KB after accounting for access-frequency promotion.

8.3 Infrastructure Validation

9 Implementation: Python MVP on SoftRoCE

The Python implementation validates the dual QP architecture on real RDMA hardware before porting to C++. All code uses pyverbs (Python bindings for rdma-core) against the SoftRoCE

Table 7: Chimera infrastructure state (March 2026).

Component	Status
RDMA device	rxe0 ACTIVE on enp69s0 (10Gbps, MTU 9000)
GPUs	3× NVIDIA GeForce RTX 3090
CUDA	12.4.131
UCX	1.16.0 (verbs, mlx5-dv, RC, DC, UD)
nvidia-peermem	Available (v550.144.03), not loaded
NIC	Aquantia AQC107 10GbE
Python / pyverbs	3.12.3 / 50.0

kernel driver (`rdma_rxe`) on Chimera's Aquantia AQC107 10GbE NIC. This section describes what the code does, how RDMA connections are established, and how testing works.

9.1 RDMA Connection Lifecycle

Every RDMA transfer requires a connected queue pair (QP). A QP starts in RESET state and must transition through three states before it can send data. Both sides must complete these transitions with each other's connection info (QP number, GID) exchanged out-of-band over TCP.

```
SETUP:
  ctx = open_device("rxe0")
  pd  = alloc_protection_domain(ctx)
  cq  = create_completion_queue(ctx, depth)
  qp  = create_queue_pair(pd, cq, type=RC)

CONNECT (both sides, with remote info exchanged over TCP):
  qp.to_init(port=1, access=LOCAL_WRITE|REMOTE_WRITE|REMOTE_READ)
  qp.to_rtr(remote_qpn, remote_gid, mtu=1024)
  qp.to_rts(timeout=14, retry=7)

REGISTER MEMORY:
  mr = register_region(pd, size, access=LOCAL_WRITE|REMOTE_WRITE)
  -- mr.buf is the virtual address, mr.lkey/rkey are access keys

TRANSFER (one-sided, no remote CPU involvement):
  post_send(qp, opcode=RDMA_WRITE, local_buf, remote_addr, remote_rkey)
  poll_cq(cq) -- blocks until NIC confirms completion
```

The key insight: `RDMA_WRITE` writes directly into remote memory using the remote address and rkey. The remote CPU never sees the transfer. This is what makes RDMA fast, and why QP isolation matters: if hot and cold traffic share a QP, the NIC processes them in FIFO order.

9.2 Dual QP Pool

The `DualQPPool` class creates two separate pools of QPs on the same protection domain. Each pool has its own completion queues, so polling one pool never touches the other.

```
class DualQPPool:
  open():
    ctx = open_device("rxe0")
    pd  = alloc_protection_domain(ctx)
    gid = query_gid(port=1, index=1)  -- link-local IPv6

    for i in 0..n_hot:
      hot_qps[i] = create_qp(pd, cq_depth=16,  max_send=16)
    for i in 0..n_cold:
      cold_qps[i] = create_qp(pd, cq_depth=256, max_send=128)

  post_write_hot(buf, remote_addr, remote_rkey, length):
    qp = hot_qps[round_robin++ % n_hot]
    t0 = now()
    post_rdma_write(qp, buf, remote_addr, remote_rkey, length)
    busy_poll(qp.cq)           -- spin until completion
    return (now() - t0) in microseconds

  post_write_cold(buf, remote_addr, remote_rkey, length):
    qp = cold_qps[round_robin++ % n_cold]
    t0 = now()
    post_rdma_write(qp, buf, remote_addr, remote_rkey, length)
    sleep_poll(qp.cq, 50us)   -- poll, sleep 50us, repeat
    return (now() - t0) in microseconds
```

Hot QPs busy-poll (spin on the CQ until a completion arrives). This burns CPU but gives minimum latency. Cold QPs sleep-poll (check, sleep 50μs, check again). This frees CPU for other work at the cost of up to 50μs added latency per poll cycle.

9.3 WFA Classifier

The classifier decides which pool handles each transfer. It examines three things: transfer size, access frequency, and inference phase.

```
classify(tensor_id, size_bytes, layer_idx):
  count = increment_access_count(tensor_id)
  phase = detect_phase(layer_idx)

  -- Size routing (primary decision)
  if size < 4KB:       queue = HOT
  elif size > 256KB:   queue = COLD
  else:
    if count >= 10:    queue = HOT    -- frequently accessed
    else:              queue = COLD   -- infrequent

  -- Phase adjustment
  if phase == PREFILL and queue == HOT and size > 2KB:
    queue = COLD       -- prefill is bulk, bias cold
```

```
  if phase == DECODE and queue == COLD and count >= 5:
    queue = HOT        -- decode is latency-sensitive, bias hot

  return queue

detect_phase(layer_idx):
  recent = last 8 layer indices seen
  if recent == [L, L+1, L+2, ...]:  return PREFILL
  if recent == [L, L, L, ...]:      return DECODE
  else:                              return UNKNOWN
```

Phase detection works by observing layer access order. Prefill sweeps sequentially through all layers (L0, L1, L2, ...). Decode repeatedly accesses the same layer as it generates tokens. This pattern is visible without any explicit signal from the inference engine.

9.4 PMP Bang-Bang Controller

The controller monitors queue depths and overrides the classifier when one pool is backing up relative to the other.

```
decide(hot_depth, cold_depth):
  -- Approximate costates by weighted queue depths
  p_hot  = alpha * hot_depth    -- alpha=2.0
  p_cold = beta  * cold_depth   -- beta=1.0

  -- Switching function
  S = p_hot * C * mu_hot - p_cold * C * mu_cold

  if S > deadband:    return COLD   -- hot congested
  if S < -deadband:   return HOT    -- cold congested
  else:               return last_decision  -- hysteresis
```

The deadband (0.1) prevents chattering: without it, the controller would flip between HOT and COLD every cycle when the queues are nearly balanced. The asymmetric weights (α=2, β=1) encode the design priority: keeping the hot queue drained matters more than keeping the cold queue drained, because hot traffic is latency-sensitive.

9.5 How RDMA Testing Works on SoftRoCE

SoftRoCE (`rdma_rxe` kernel module) implements the RDMA verbs API in software over any Ethernet NIC. It is not hardware RDMA: the kernel handles all QP state, packet construction, and completion notification. Latencies are higher than hardware RDMA (tens of microseconds vs single-digit), but the API is identical. Code that works on SoftRoCE works on ConnectX hardware with no changes.

Loopback testing. Two `DualQPPool` instances on the same machine, both using `rxe0`. QP info exchanged in-process, all QP pairs connected, then RDMA writes measured for completion latency.

```
pool_a = DualQPPool("rxe0", n_hot=2, n_cold=2)
pool_b = DualQPPool("rxe0", n_hot=2, n_cold=2)
pool_a.open();  pool_b.open()

pool_a.connect_all(pool_b.get_local_info())
pool_b.connect_all(pool_a.get_local_info())

send_buf = pool_a.register_buffer("send", 1MB)
recv_buf = pool_b.register_buffer("recv", 1MB)

for i in 0..1000:
  if i is even:
    pool_a.post_write_hot(send_buf, recv_buf.addr,
                          recv_buf.rkey, 256B)
  else:
    pool_a.post_write_cold(send_buf, recv_buf.addr,
                           recv_buf.rkey, 1MB)
```

Cross-node testing. For Chimera ↔ Cerberus, each machine runs one pool. QP info exchanges over TCP (192.168.1.x management network), RDMA traffic flows over the 10.0.0.x subnet (direct 10GbE cable). Both machines need SoftRoCE active and an IPv4-mapped GID in the RDMA GID table (populated by adding an IPv4 address to the RDMA-facing interface).

Lessons learned.

- pyverbs on Ubuntu 24.04 uses property setters for `QPAttr`, `set_wr_rdma(rkey, addr)` for RDMA work requests, and `cq.poll()` returns `(count, list)`. These differ from rdma-core documentation and required iterative testing.
- SoftRoCE requires a routable GID for cross-node connections. Link-local IPv6 works for loopback but not between machines. Adding an IPv4 address populates an IPv4-mapped GID at a higher index.
- The `rdma_rxe` module enters a zombie state if processes exit without closing resources, requiring a reboot. Production code must handle cleanup carefully.
- With synchronous single-threaded SoftRoCE, head-of-line blocking does not manifest because each operation completes before the next starts. The dual QP benefit requires concurrent producers or hardware CQ offload.

10 Next Steps

1. Cross-node benchmark between Chimera and Cerberus over real 10GbE (blocked on k8s firewall rules for the 10.0.0.x RDMA subnet).
2. Concurrent workload benchmark: multi-threaded producers to demonstrate actual head-of-line blocking elimination under contention.
3. UCX comparison: run `ucx_perftest` at protocol boundary sizes to measure the eager/RNDV cliff, compare against our smooth latency curve.

4. vLLM integration: implement `ScuffedRdmaConnector` as a `KvConnector` backend, replacing NixlConnector's transport path.

5. KV cache compression: integrate LoRC/KVTC/TurboQuant to reduce transfer sizes before RDMA.

6. Rust transport layer: `tch-rs` (Rust bindings for libtorch) could replace the Python pyverbs layer with a zero-overhead Rust transport that calls `ibv_post_send` through `rdma-core-sys` FFI bindings. This would eliminate Python's GIL as a bottleneck for concurrent QP polling and enable the lock-free SPSC ring buffer design planned for the hot QP path.

7. Triton kernel integration: custom kernels (K-Search [7]) for fused attention + RDMA notification, overlapping KV computation with RDMA writes to pipeline prefill and transfer.

8. Advisor alignment: PMP formalization with Dr. Danesh; WFA decidability with Dr. Suchy.

References

[1] J. Austin et al., "How to scale your model," Google DeepMind, 2025. `https://jax-ml.github.io/scaling-book/`

[2] C. Zhang, J. Cheng, and Q. Li, "An optimal control view of LoRA and binary controller design for vision transformers," *Proc. ICML*, 2024. arXiv:2406.06958.

[3] L. S. Pontryagin, *Mathematical Theory of Optimal Processes*. Routledge, 2018.

[4] R. Bellman, I. Glicksberg, and O. Gross, "On the 'bang-bang' control problem," *Q. Appl. Math.*, vol. 14, no. 1, pp. 11–18, 1956.

[5] D. Liberzon, *Calculus of Variations and Optimal Control Theory*. Princeton Univ. Press, 2011.

[6] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," arXiv:2106.09685, 2021.

[7] S. Cao and I. Stoica, "K-Search," arXiv:2602.19128, 2025.

[8] R. Zhang et al., "LoRC: Low-rank compression for LLMs KV cache with a progressive compression strategy," arXiv:2410.03111, 2024.

[9] M. Staniszewski and T. Łańcucki, "KV cache transform coding for compact storage in LLM inference," arXiv:2511.01815, 2024.

[10] A. Zandieh, M. Daliri, M. Hadian, and V. Mirrokni, "TurboQuant: Online vector quantization with near-optimal distortion rate," Google Research, arXiv:2504.19874, 2025. Uses PolarQuant (arXiv:2502.02617) and QJL.

[11] NVIDIA, "Dynamo: Open-source inference optimization framework," GTC 2025.

[12] P. Tillet, H. T. Kung, and D. Cox, "Triton," *Proc. MAPL*, 2019.

[13] R. Dyro, "Pallas-Triton kernels and kernel auto-tuning," 2025. `https://robertdyro.com/articles/pallas-triton_kernels/`

[14] dhmnr, "Introduction to PTX optimization," 2025. https://dhmnr.sh/posts/intro-to-ptx-optimization/

[15] L. Chen et al., "RDMA point-to-point communication for LLM systems," arXiv:2510.27656, 2025.

[16] Mistral AI, "Heaps do lie: Debugging a memory leak in vLLM," 2025.

[17] Red Hat, Google, IBM, "llm-d v0.5: Sustaining performance at scale," 2026.

[18] NVIDIA, "HPC-X v2.15: UCX Framework Library."

[19] AMD, "MaxText-Slurm," ROCm Blog, 2025.

[20] vLLM, "vLLM TPU: Unified backend supporting PyTorch and JAX on TPU," 2025.

[21] vLLM Project, "PR #34861: Elastic expert parallelism," 2026.

[22] L. Chen, "Harnessing 3200 Gbps network (1): RDMA and EFA," 2024.

[23] R. Dyro, "torch2jax," 2025. https://robertdyro.com/torch2jax/roadmap/

[24] SemiAnalysis, "Nvidia Blackwell perf TCO analysis," 2025.

[25] R. Sharoon, "Paxos vs Raft vs ZAB," Medium, 2024.

[26] N. Shazeer, "Fast transformer decoding: One write-head is all you need," arXiv:1911.02150, 2019.

[27] J. Ainslie et al., "GQA: Training generalized multi-query transformer models from multi-head checkpoints," arXiv:2305.13245, 2023.

[28] D. Benveniste, "How to improve decoding latency with faster self-attention mechanisms," The AiEdge Newsletter, 2025.

[29] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," *Proc. SOSP*, 2023.

[30] Apple, "macOS 26.2: RDMA over Thunderbolt," 2025. HN: https://news.ycombinator.com/item?id=46248644

[31] D. Barak, "Tips and tricks to optimize your RDMA code," RDMAmojo, 2013. https://www.rdmamojo.com/

[32] B. Rothenberger et al., "ReDMArk: Bypassing RDMA security mechanisms," *Proc. USENIX Security*, 2021.

[33] FlashInfer, "Kernel library for LLM serving," 2025. https://github.com/flashinfer-ai/flashinfer